

EdgeDispatch

An Adaptive Inference Bridge

Local Classification, Local Execution, Selective Escalation

Dolwin Fernandes

Master of Artificial Intelligence

La Trobe University

Supervisor: Dr. Phu Lai

CSE5TSB - Thesis

October 2026

Submitted in fulfilment of the thesis requirements of CSE5TSB,
Department of Computing, Engineering and Mathematical Sciences,
La Trobe University, October 2026.

Contents

Abstract	iii
1 Introduction	1
1.1 Background and motivation	1
1.1.1 Classifying and completing work locally	1
1.1.2 Why local execution is practical	2
1.1.3 Tasks that motivate selective escalation	2
1.2 Research problem and objectives	3
1.3 Contributions and scope	4
1.4 Study approach and thesis organisation	4
2 Research Problem and Foundations	6
2.1 Execution decision and task context	6
2.2 Data and experimental populations	6
2.3 Evaluation measures and cost	7
3 Literature Review	8
3.1 External evidence, context and MCP	8
3.2 Hybrid inference and learned allocation	9
3.3 Local-model capability and adaptation	9
3.4 Evaluating tool use and complete answers	10
3.5 Synthesis and research gap	11
4 Methodology and Implementation	13
4.1 System architecture	13
4.1.1 Harness components and deployment	13
4.1.2 From a user question to a completed answer	15
4.1.3 MCP connections, history and streamed updates	16
4.2 Synthetic source environment	16
4.3 Determining local capability	18
4.3.1 Synthetic tasks and matched execution	18
4.3.2 Capability labels for the final airline dataset	21
4.4 SFT training procedure	24
4.5 Prospective classifier test	25
4.6 Three-arm answer-quality evaluation	25

5	Results, Analysis and Discussion	27
5.1	Evaluation overview and task design	27
5.2	Final classifier results	27
5.2.1	Classification measures	28
5.2.2	Escalation and local retention	29
5.3	Delivery and answer quality	30
5.4	Complete hard-question experiment	31
5.5	Remote execution and API-cost savings	32
6	Conclusion and Future Work	34
6.1	Conclusions	34
6.2	Future work	35
	References	37

Abstract

Many service requests can be completed by a compact local model using records and tools available on the user’s device. The challenge is to identify that work before committing to local or remote execution. We address this problem with EdgeDispatch, a local-first agent system whose *Adaptive Inference Bridge* connects a capability-based execution decision to task completion. Requests selected for local handling are answered on the device, avoiding a remote execution API call. Other requests are escalated to a remote executor.

We implement the system with a high capable small language model - LFM2.5-2.6B with llama.cpp as the inference engine. Liquid AI reports tool-use and instruction-following results competitive with models around three times its size, supporting its use in agent workflows with MCP tools. We assess the local executor through recorded task attempts and train a LoRA adapter to predict LOCAL or ESCALATE from reviewed outcomes. The adapter is enabled for classification and disabled when the same local base executes retained tasks. A shared harness manages conversation history, MCP connections and the selected model’s tool-use loop.

We evaluate the decision adapter separately from complete answers, then compare local-only, remote-only and adaptive execution in a simulated airline environment. The classifier identifies most reviewed local execution failures while retaining useful local work. On the mixed workload, EdgeDispatch delivers every answer, improves judged answer quality over local-only execution and reduces remote execution API expenditure compared with remote-only execution. We assess correctness and faithfulness using Phoenix’s native LLM evaluators.

The contribution is a method for learning an execution decision from the assessed capability of its local executor and evaluating the resulting service. The model, available tools and observed outcomes define the decision target, providing a basis for adapting the approach to other local configurations.

Keywords: Adaptive Inference Bridge, local-first agents, local execution, supervised fine-tuning, LoRA, Model Context Protocol, LLM evaluation.

Chapter 1

Introduction

1.1 Background and motivation

Can this request be completed here, by the local model and its available tools? This is the question at the centre of EdgeDispatch. The system uses a compact, fine-tuned local language model to assess a request and, when it selects **LOCAL**, to perform the work itself locally: call tools, read the returned evidence, apply the relevant instructions and produce the final answer. A request that requires remote execution is marked **ESCALATE**. The operating principle is *classify locally, execute locally when selected, and escalate when needed*.

We call this design an *Adaptive Inference Bridge*. It connects an on-device agent’s existing capabilities to a remote executor for more demanding work. By completing suitable tasks on the user’s device, we aim to avoid the input and output token charges of remote execution. We investigate how much useful work can be completed this way while identifying requests that warrant escalation.

1.1.1 Classifying and completing work locally

We centre the execution decision on local capability: can a small language model running on the device complete this task with the available tools? The decision selects an execution path. On the local path, the same base model calls the tools, interprets the evidence and returns the answer.

We use LFM2.5-2.6B in both roles. A small execution-decision LoRA adapter is enabled for the LOCAL/ESCALATE prediction. For a retained task, the adapter is disabled and the base model’s existing agentic capabilities produce tool calls and the answer with llama.cpp inference. Supervised fine-tuning (SFT) teaches the execution decision based on learned capabilities of the model, while the base model supplies task-solving capability. The surrounding harness manages tool access, conversation state and progression from assessment to a final answer.

We learn the capability assessment from observed local execution outcomes. This gives the decision a concrete target: retain work that the configured local executor

can complete and escalate work beyond its predicted capability.

At enterprise scale, completing a share of daily work locally can produce substantial cumulative API-cost savings. Consider an airline or bank in which 20-25% of AI-assisted tasks can be completed on employees’ devices. This reduces the number of remotely executed tasks by the same proportion. If those tasks represent 20-25% of the otherwise remote token expenditure, the execution API saving is phenomenal especially across thousands of users and repeated daily requests. Retaining this work locally makes a meaningful difference to recurring expenditure. The saving depends on the token usage of the tasks completed locally, which we measure directly in our experiments.

1.1.2 Why local execution is practical

Compact models and efficient inference software make on-device agents practical on suitably equipped consumer laptops, desktops and GPUs. LFM2.5-2.6B combines a compact 2.6-billion-parameter model with function-calling support and agentic capabilities. Liquid AI reports its performance on function-calling and interactive tool-use benchmarks, including BFCLv4 and ToolSandbox (Liquid AI, 2026). These characteristics make it a useful model for studying local execution of tasks that require tools, retrieved records and several steps of reasoning. Chapter 3 develops the model-selection rationale and the relevant benchmark results.

The serving platform makes this capability accessible locally. Apple silicon M4 and M5 systems provide unified memory, while llama.cpp supports quantised inference and acceleration through Metal, Accelerate and ARM NEON (Apple, 2024, 2025, ggml-org, 2026). We use llama.cpp to serve the local model and its decision adapter, and measure the resulting remote API-cost saving alongside answer quality.

1.1.3 Tasks that motivate selective escalation

We take a customer-service request as an example of why local capability must be assessed against the complete task. Asking for the fare type of one named airline booking requires a direct retrieval and a short response. Asking for an optimal combination of bookings under a budget, with customer-coverage and tie-breaking requirements, demands retrieval followed by constrained reasoning. Both requests may expose the same tools, yet place very different demands on the executor. Assessing actual outcomes makes this difference visible and provides evidence for the execution decision.

Retrieval-augmented generation established an influential way to connect generation to external evidence (Lewis et al., 2020). The Model Context Protocol (MCP)

provides a common interface between an application and tool-serving systems ([Model Context Protocol, 2025a](#)). The Adaptive Inference Bridge uses both these methods. It first assesses the request and available tools to decide whether the local model can undertake the task reliably. The selected executor then uses the MCP interfaces to obtain records and policies and produce the answer. Remote inference supplies an escalation path for work beyond the predicted local capability.

1.2 Research problem and objectives

Our research problem is how to identify work that the local executor can complete and recognise when remote assistance is appropriate. We use a **local System 1-style classifier based on assessed capability**. Here, System 1-style describes the short decision stage that precedes full task execution. A general-purpose remote model could be prompted to classify a request, but that prompt alone would not establish how this particular local executor performs with its tools. We instead train the local classifier on reviewed outcomes from the configured executor. Its prediction concerns that executor’s capability to complete the task, rather than the request’s general topic or intent. Both classification and retained execution take place on the local device.

We pursue four objectives. First, we implement an Adaptive Inference Bridge in which the same local base supports capability assessment and complete task execution. Second, we train an execution-decision adapter using evidence about local reliability. Third, we compare local-only, remote-only and adaptive execution using the same questions, tools and source data. Fourth, we connect reported results to complete answers, tool interactions and evaluator explanations.

The resulting research questions are:

1. **RQ1:** How effectively can the SFT execution-decision adapter identify reviewed local failures for escalation and retain tasks that the local executor can complete?
2. **RQ2:** How do answer delivery, correctness and faithfulness differ between local-only, remote-only and EdgeDispatch execution?
3. **RQ3:** What reduction in remote execution and recorded execution API cost is achieved?
4. **RQ4:** How do the execution harness and evaluation records support a traceable assessment of the system?

We evaluate execution decisions and final answers separately. The first measures whether work is assigned appropriately, while the second measures whether the

selected executor completes it correctly and uses the evidence faithfully. Remote answer quality depends on the selected remote model, and a more capable remote executor may improve results. Accordingly, the same answer-quality measures are applied to both local and remote outputs.

1.3 Contributions and scope

Our first contribution is an implemented Adaptive Inference Bridge that uses one locally served base model in two roles: the adapter-enabled model assesses local capability, and the adapter-disabled model performs the retained task. The second is a reproducible SFT procedure using around 1,000 synthetic decision examples, labelled according to reviewed local execution outcomes. The third is a comparative evaluation of the local-only, remote-only and adaptive paths, reporting both answer delivery and judged answer quality. The fourth is a traceable experimental record connecting reported totals to individual answers, source interactions and evaluator outputs.

Our study brings together learned execution decisions, low-rank adaptation and tool-using agents. The reusable element is the method for relating a model’s local capability to the execution decision. Applying the architecture to another local model requires assessing that model on representative tasks, deriving suitable decision targets and evaluating the resulting local and remote paths.

We study simulated airline customer service, including bookings, fare policies, refunds and support cases. Three MCP servers expose nine read-only tools over synthetic records. This environment supports direct lookups as well as demanding tasks with linked evidence and constraints. It gives each execution path access to the same business information and allows complete answers to be checked against recorded sources.

1.4 Study approach and thesis organisation

Figure 1.1 summarises our research workflow. We begin by defining the tool environment and gathering evidence of local execution. We use the reviewed outcomes to train the execution-decision adapter, assess checkpoints on the development set and evaluate the selected candidate on a separate prospective classifier set. We then compare complete execution through the local-only, remote-only and EdgeDispatch paths. Correctness, faithfulness and remote execution cost show how the decisions affect the delivered service.

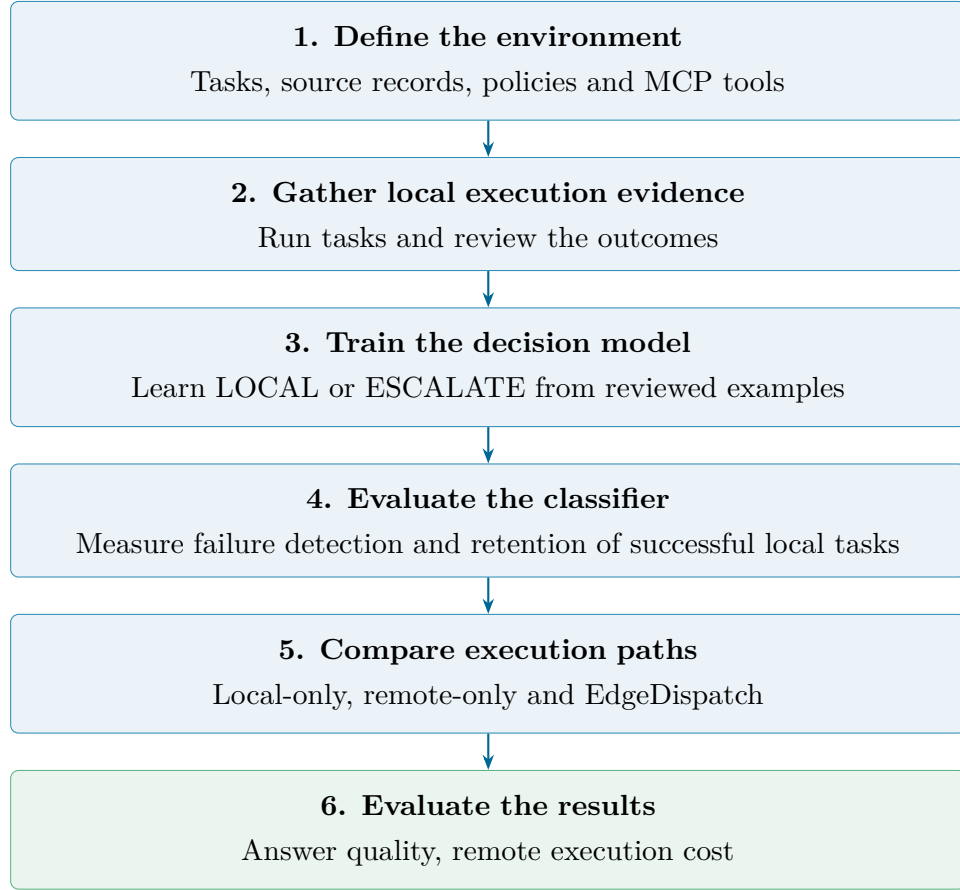


Figure 1.1: Research workflow from local execution evidence to evaluation of the complete system.

Chapter 2 introduces the local-versus-escalate decision, the experimental populations and the evaluation measures. Chapter 3 connects external evidence and MCP, learned model allocation, local-model capability, adaptation and agent evaluation to the research problem. Chapter 4 explains the implementation, training procedure and experimental controls. Chapter 5 presents classifier and system results and explains their practical significance. Chapter 6 answers the research questions and identifies the next experiments.

Chapter 2

Research Problem and Foundations

2.1 Execution decision and task context

Retrieval supplies external information, while execution allocation selects the model that will use it to complete a task. EdgeDispatch connects these functions by assessing local capability, then answering through the selected path.

In our airline environment, retrieving a booking gives the executor facts, while determining refund eligibility also requires applying a policy. Support cases may contain claims later corrected by payment records. The executor must reconcile this evidence and distinguish a support agent’s statement from an authoritative rule or payment event. Retrieval and reasoning therefore contribute separately to completion.

We represent a user request and its available context, including the conversation and tools, as x . The execution-decision model returns

$$d_{\theta}(x) \in \{\text{LOCAL}, \text{ESCALATE}\}. \quad (2.1)$$

LOCAL retains the task for completion by the local model. **ESCALATE** forwards it to the configured remote executor. The learning target is whether the local executor can reliably complete the task with the available tools and evidence. This capability boundary is assessed for the local small language model LFM used in the system.

2.2 Data and experimental populations

Table 2.1 distinguishes the training, classifier and answer-quality populations. Decision contexts capture a request at a particular point in a conversation. Around 1,000 training contexts span 210 source conversations. Answer-quality items are questions executed under all three study conditions.

Synthetic bookings, policies and support histories provide repeatable evidence against which we can check each answer.

Table 2.1: Dataset roles and resolved sample counts.

Population	Count	Purpose
Training decision contexts	1,020	Learn LOCAL/ESCALATE, with 558 LOCAL and 462 ESCALATE.
Development contexts	199	Checkpoint assessment, with 105 LOCAL and 94 ESCALATE.
Prospective classifier contexts	197	Selected-candidate assessment, with 86 LOCAL and 111 ESCALATE.
Hard answer-quality questions	100	Stress evaluation of three execution arms.
Mixed answer-quality questions	50	26 straightforward plus 24 questions drawn from the hard set.

2.3 Evaluation measures and cost

We compare classifier predictions with reviewed local outcomes, reporting overall and balanced accuracy, precision and recall for both decisions, and ESCALATE F1. ESCALATE is the positive class. Chapter 5 explains these measures using the test counts. The three-arm experiments assess answer quality, local completion and remote API cost.

Delivery rate uses all attempted questions as its denominator. Correctness and faithfulness pass rates use answers successfully scored for each metric. Reporting passes per attempted question also shows how delivery and answer quality contribute to overall completion.

Correctness assesses factual accuracy, completeness and logical consistency. Faithfulness assesses support for the answer’s claims in the supplied evidence. For example, faithfully quoting prices does not establish that a selected booking combination satisfies the user’s requirements.

We calculate remote execution API cost from recorded input, output and cached-token usage under the study’s fixed pricing assumptions. We compare EdgeDispatch’s remote execution cost with remote-only execution over the same question set. If C_R is the remote-only cost and C_E is the EdgeDispatch cost, the saving is

$$\text{Remote execution cost saving (\%)} = \frac{C_R - C_E}{C_R} \times 100. \quad (2.2)$$

We report this comparison as a percentage of the remote-only execution cost, allowing the saving to be read independently of the absolute expenditure of the sample.

Chapter 3

Literature Review

3.1 External evidence, context and MCP

Many useful language-model tasks depend on information held outside the model, such as customer records, booking details and current policies. Retrieval-augmented generation established the value of supplying external evidence at inference time (Lewis et al., 2020). In our system, models obtain this evidence through tools that retrieve structured records and policy text. A booking question can therefore be answered using the details of the booking and the rules that apply to it.

Tool-using agents extend this process by deciding which information to obtain as a task develops. ReAct interleaves reasoning and actions, allowing retrieved observations to inform subsequent steps (Yao et al., 2023). Toolformer studies learning when and how to use APIs (Schick et al., 2023). These approaches establish ways for a model to act on external information. We build on an existing model’s tool-use capability and study the preceding decision about whether that local model should undertake the task.

How information is presented also affects its use. Liu et al. (2024) show that models can be sensitive to the position of relevant information within a long context, while Shi et al. (2023) find that irrelevant information can distract models during reasoning. Successful retrieval therefore supplies only part of what an agent needs. It must also interpret the returned information in relation to the request. This distinction matters for our support-case questions, where an earlier statement and a later payment record may describe different stages of the same case.

MCP supplies a common interface for discovering and calling tools (Model Context Protocol, 2025a,b). A consistent interface lets the local and remote executors access the same sources, but it does not itself determine whether either model can solve a task. We therefore assess local capability in the configured tool environment and retain the conversation and returned evidence for analysis. MCP standardises access, while the execution records show how the model uses that access.

3.2 Hybrid inference and learned allocation

Efficient language-model systems allocate work across models with different capabilities and costs. [Chen et al. \(2023\)](#) propose FrugalGPT, a cascade that learns combinations of models for different queries. Its central contribution is to treat model selection as an optimisation problem involving both expenditure and answer accuracy. This establishes a useful comparison for EdgeDispatch: selective use of a remote model can be assessed against sending every request to that model.

[Ding et al. \(2024\)](#) develop a hybrid approach that allocates queries between small and large models using predicted difficulty and a desired quality level. The quality target can be adjusted at inference time, making the allocation responsive to application requirements. [Ong et al. \(2025\)](#) train RouteLLM using preference data, learning which of a stronger and a weaker model to select. Their results also examine transfer to model pairs outside the training configuration. These studies show that learned allocation can manage the relationship between cost and quality through different forms of supervision.

The supervision determines what an allocation decision means. A preference label identifies which answer is favoured in a comparison. A local-capability label asks whether one specified executor completed the task to the required standard. For our setting, the latter makes the decision directly interpretable: a LOCAL prediction means that the configured local model is expected to complete the request with its tools. We derive this target from reviewed local execution outcomes, including the evidence used and the requirements of the task.

We also place classification on the device. A general-purpose remote model can make a classification when prompted, while a capability-trained decision model has supervision tied to the executor it assesses. The location of inference and the source of supervision are separate design choices. EdgeDispatch combines local inference with capability-specific supervision, then uses the same local base to execute retained work. We assess the resulting policy in two ways: classifier measures test its agreement with reviewed outcomes, and complete execution tests the answers and remote API cost that follow from its decisions.

3.3 Local-model capability and adaptation

Compact models make selective local execution practical when they combine manageable model size with useful tool and agent capabilities. Liquid AI describes LFM2.5-2.6B as an on-device model with agentic post-training and function-calling support. In its published comparison, LFM scores 56.88 on BFCLv4 and 77.83 on

ToolSandbox. Liquid AI also reports higher LFM scores on IFBench, Multi-IF and IFStruct in that comparison (Liquid AI, 2026). These results provide a concrete rationale for selecting a compact model to follow instructions, invoke functions and use returned evidence in an MCP-based agent workflow. It gives us state of the art capabilities which can be compared to models nearly 3x its size with means significant memory savings as well.

The published benchmarks inform model selection, while our experiments assess capability in the actual application environment. Straightforward lookups and bounded policy decisions provide opportunities for local completion. Requests involving several linked records or interacting constraints test a different set of reasoning demands. Evaluating both kinds of work lets us connect the model’s general tool-use strengths to the specific tasks that the decision adapter must recognise.

LoRA learns small, low-rank updates while keeping the base model weights fixed (Hu et al., 2021). We use supervised fine-tuning to teach a LoRA adapter the LOCAL/ESCALATE decision. The adapter is enabled when assessing the request and disabled when the local base executes retained work. This separation is central to the experiment: we train the execution decision and evaluate the base model’s existing task-solving capability. The adapter adds compact decision weights without requiring a second full local model.

Using another local model would change the capability being assessed. The corresponding training labels would need to describe that model’s performance in the target environment. This is why we treat LFM2.5-2.6B as a concrete implementation of a reusable method, rather than treating its decision labels as universal descriptions of task difficulty.

3.4 Evaluating tool use and complete answers

Agent benchmarks show why evaluation must examine more than whether a tool call is syntactically valid. Yao et al. (2024) introduce τ -bench, in which an agent interacts with a simulated user, domain policies and API tools. They evaluate the final database state against a goal and measure reliability over repeated trials. ToolSandbox includes stateful tool execution and dependencies between tools, with checks at intermediate and final milestones (Lu et al., 2024). Both make the sequence and consequences of actions part of the assessment.

Our read-only environment requires a different observable outcome. Since a completed task produces an answer rather than a changed booking or payment, we evaluate what the answer says and how it uses the retrieved evidence. The benchmark

literature nevertheless motivates preserving the full trajectory. A final response can be interpreted more precisely when the retrieved records, applied policy and intermediate decisions are available. The distinction also explains why our classifier test and answer-quality experiments answer separate questions. Correct allocation is one step towards successful service, while the completed answer establishes the outcome of the chosen path.

Automated judges provide a practical way to assess natural-language answers across a controlled set of attempts. [Zheng et al. \(2023\)](#) find substantial agreement between strong language-model judges and human preferences, while identifying position, verbosity and self-enhancement biases. This supports using explicit evaluation criteria and retaining explanations so that individual judgements can be examined. Agreement reported on their benchmarks is evidence for the approach, rather than a validation of every judgement in a new domain.

We use Phoenix’s native correctness and faithfulness evaluators. Correctness assesses whether an answer is accurate, complete and consistent with the task, while faithfulness assesses whether its claims are supported by the supplied context ([Arize AI, 2026a,b](#)). These dimensions distinguish selecting the correct bookings from accurately quoting their recorded prices. We preserve the final answers, evaluator inputs, scores and explanations, and use source checks to investigate specific requirements.

3.5 Synthesis and research gap

The literature supplies complementary parts of a local-first agent system. Tool-use research explains how models act on external evidence. Allocation research relates model choice to cost and quality. Adaptation methods make a compact decision component feasible, and agent evaluation shows how to assess the consequences of actions. Table [3.1](#) summarises the implications for our design.

Table 3.1: Literature synthesis and implications for EdgeDispatch.

Research strand	Established contribution	Application in EdgeDispatch
ReAct / Toolformer	Models interact with tools during a task.	Use an existing tool-capable local executor to complete retained requests.
RAG / context studies	External evidence and its presentation affect answers.	Retrieve task evidence and preserve its use in the execution trajectory.
FrugalGPT / Hybrid LLM / RouteLLM	Learned allocation manages model cost and quality.	Supervise the decision using reviewed outcomes from the configured local executor.
LoRA	Low-rank adaptation learns compact updates to a frozen base.	Enable decision weights for classification and disable them for local execution.
τ -bench / ToolSand-box	Task outcomes and tool dependencies matter to agent evaluation.	Record complete tool trajectories and assess the delivered answer.

The specific question we address is how to connect **assessed local capability** to a decision that produces useful local work. The target is whether this model, with these tools and the available context, can complete the request reliably. Reviewed execution outcomes supply that target. This gives the classifier a direct relationship to the executor’s observed behaviour, which is more specific than a general judgement about the request’s topic or apparent difficulty.

We investigate this connection through an integrated system. The local model uses the adapter to select LOCAL or ESCALATE, and the base model completes retained tasks through the same harness used during capability assessment. We evaluate the decision policy alongside delivered answer quality and remote API cost. This addresses an applied research gap at the intersection of capability assessment, local classification and complete tool-based execution. Chapter 4 describes the implementation and experiments through which we examine it.

Chapter 4

Methodology and Implementation

We combine implementation, supervised adaptation and comparative evaluation to examine how a local capability decision affects completed answers and remote execution. We first establish a common tool environment, then train the execution-decision adapter while keeping the local executor unchanged. We assess the classifier against reviewed local outcomes and compare complete answers across local-only, remote-only and adaptive execution. The classifier test shows whether the adapter recognises tasks requiring escalation. The three-arm comparison establishes how those decisions affect the service delivered to the user.

4.1 System architecture

Figure 4.1 shows the Adaptive Inference Bridge implemented by EdgeDispatch. Its local component both classifies and executes, with the remote component providing an escalation path. The FastAPI harness/server receives user requests, coordinates conversations, manages MCP connections through the OpenAI Agents SDK and constructs the execution-decision input. JSONL records preserve conversation and tool activity, while Redis supports coordination.

We serve LFM2.5-2.6B through llama.cpp with Q8 weights and a separate F16 LoRA adapter. The same local model therefore supports classification and execution. Here, base means the post-trained LFM2.5-2.6B checkpoint before our adapter. We activate the adapter for classification and set its scale to zero for local task execution, preserving the unchanged executor. The remote path uses `gpt-5.6-luna` through the Responses API. Both executors receive the request, conversation and available tools, then choose the retrieval sequence needed to answer. The harness also provides a remote safeguard for an unrecognised decision label and a fallback when local execution cannot complete.

4.1.1 Harness components and deployment

We implement the harness as a React browser interface connected to a FastAPI server. The browser manages the visible conversation, question entry and the

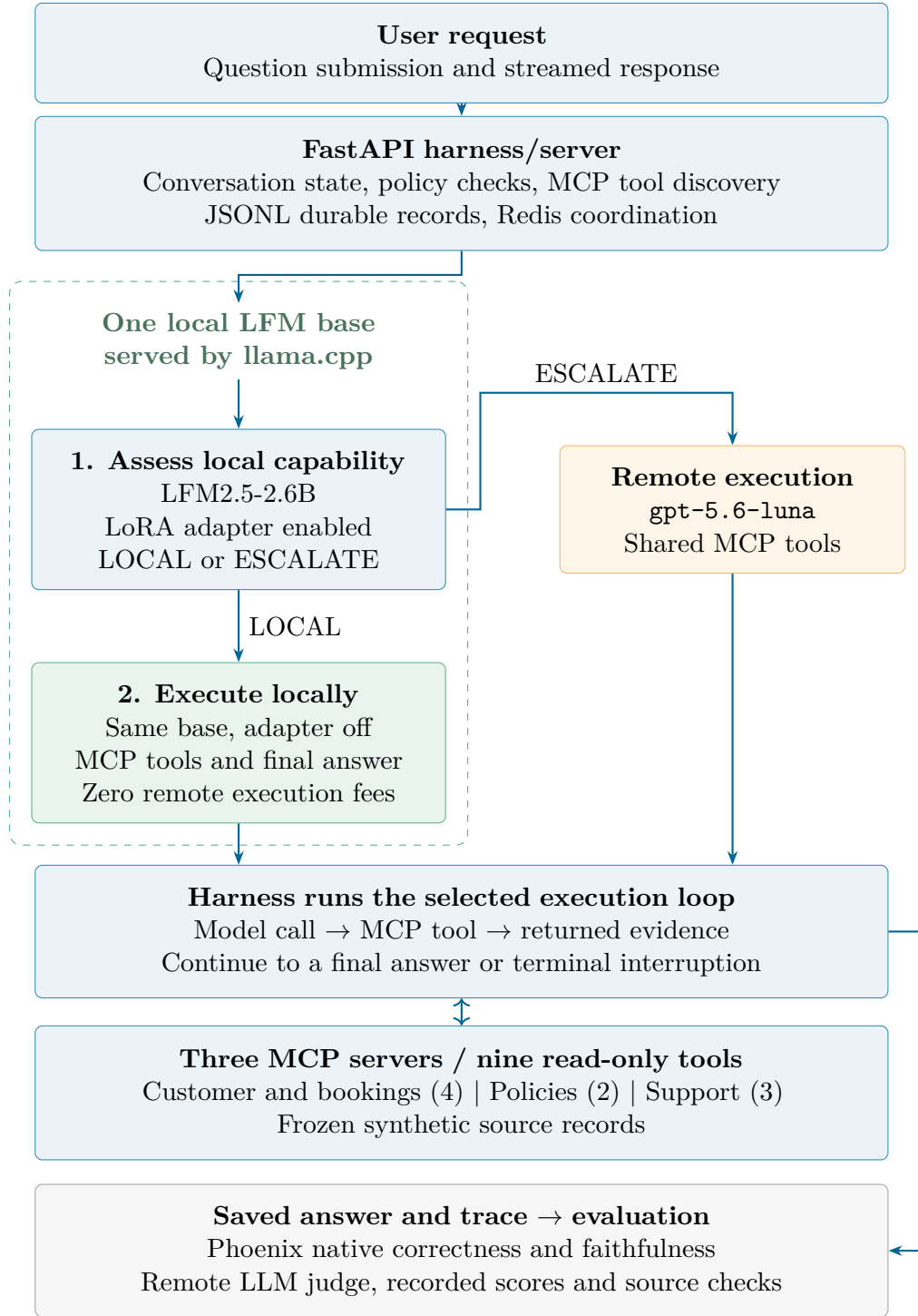


Figure 4.1: The Adaptive Inference Bridge. The same local LFM base assesses the request and executes retained tasks through llama.cpp. Both execution paths use the MCP environment. Phoenix provides the evaluation path.

streamed answer. FastAPI owns the execution workflow, including request validation, conversation policy, local capability assessment, model calls and MCP tool invocation. Keeping these responsibilities in the server gives both execution paths the same handling of tools, history and recorded outcomes.

Our local setup runs the React/Vite interface and FastAPI as host processes, alongside the llama.cpp model server. Docker Compose supplies Redis for conversation coordination and Phoenix for trace and evaluation storage. Each container uses a persistent volume, and its published port is bound to the local machine. The FastAPI process starts the three synthetic MCP servers as Python subprocesses. The remote model is accessed through the configured API when the selected path requires escalation. This arrangement places the application, local model, conversation record and MCP data on the workstation while retaining a remote execution option.

We configure model endpoints, the selected adapter, data locations and remote access through environment settings. The frontend sends its API requests to the backend, which loads the durable JSONL conversation log and establishes the supporting connections at startup. The application is run with one backend worker because that process owns this file. A readiness endpoint reports the availability of Redis, the local model and connected MCP servers. These checks give the interface a direct view of the components needed to accept and execute a request.

4.1.2 From a user question to a completed answer

Figure 4.2 follows a request through the harness. We first create a conversation identifier. When the user submits a question, the browser attaches a separate request identifier and sends the question to FastAPI. The server records a turn containing that request and the settings used for it. Repeated delivery of the same request identifier resolves to the existing turn, so reconnecting the browser does not create another execution of the same submitted question.

The runtime then acquires a Redis lease for the conversation. A lease gives one active execution ownership of that conversation and is renewed while the task is running. This prevents concurrent work from mixing messages within the same conversation. The execution checks ownership before proceeding with model or tool operations. At completion or cancellation, it releases the lease. Redis therefore coordinates who may act on a conversation, while the JSONL log preserves what happened.

Next, the harness reconstructs the conversation from its recorded events and discovers the currently available MCP tools. For the first question, this context contains the new user request and the tool interface. For a follow-up, it also includes earlier assistant messages and returned tool evidence. The classifier receives that context

and returns LOCAL or ESCALATE. In our evaluated adaptive condition, each new request is reconsidered.

The OpenAI Agents SDK runs the model-and-tool loop. The selected model requests a tool when it needs source information, the harness obtains the result, and that result is added to the context for the next model call. The loop ends when the executor produces its final answer.

4.1.3 MCP connections, history and streamed updates

The FastAPI startup lifecycle opens the MCP connections through the Agents SDK and keeps them available for execution. Our synthetic environment uses the SDK’s standard-input/output transport to communicate with the three Python MCP processes. Tool discovery retrieves their names, descriptions and input schemas. The harness wraps these definitions as callable agent tools, validates the requested arguments and invokes the matching MCP server. The returned records are passed back to the executor as tool results. The records are synthetic, but discovery and invocation use the MCP interfaces.

For example, a question about a named booking can lead to a booking lookup on the customer-and-bookings server. If answering also requires an upgrade rule, the executor retrieves that rule from the policy server. The harness records each tool name, its arguments and the returned content. The model can then combine the booking facts and policy in its answer. This same mechanism supports more involved requests that link a customer history, support case and refund record across servers.

The durable record is an append-only JSONL event log. Events have ordered sequence numbers and conversation and turn identifiers. They capture the question, decision, tool activity, model usage, answer updates and final outcome. The harness reconstructs conversation history from these records, so it remains available after the browser closes. Each tool result stays associated with its call and the answer that used it.

The browser receives progress through server-sent events. It can display the chosen execution path, tool activity and answer text as the turn progresses. The user’s original question, the local or remote execution path and the final score go through one recorded workflow.

4.2 Synthetic source environment

We use a synthetic airline service desk for the answer-quality experiments. Customer histories link to bookings with fare and ticket attributes. Support cases contain

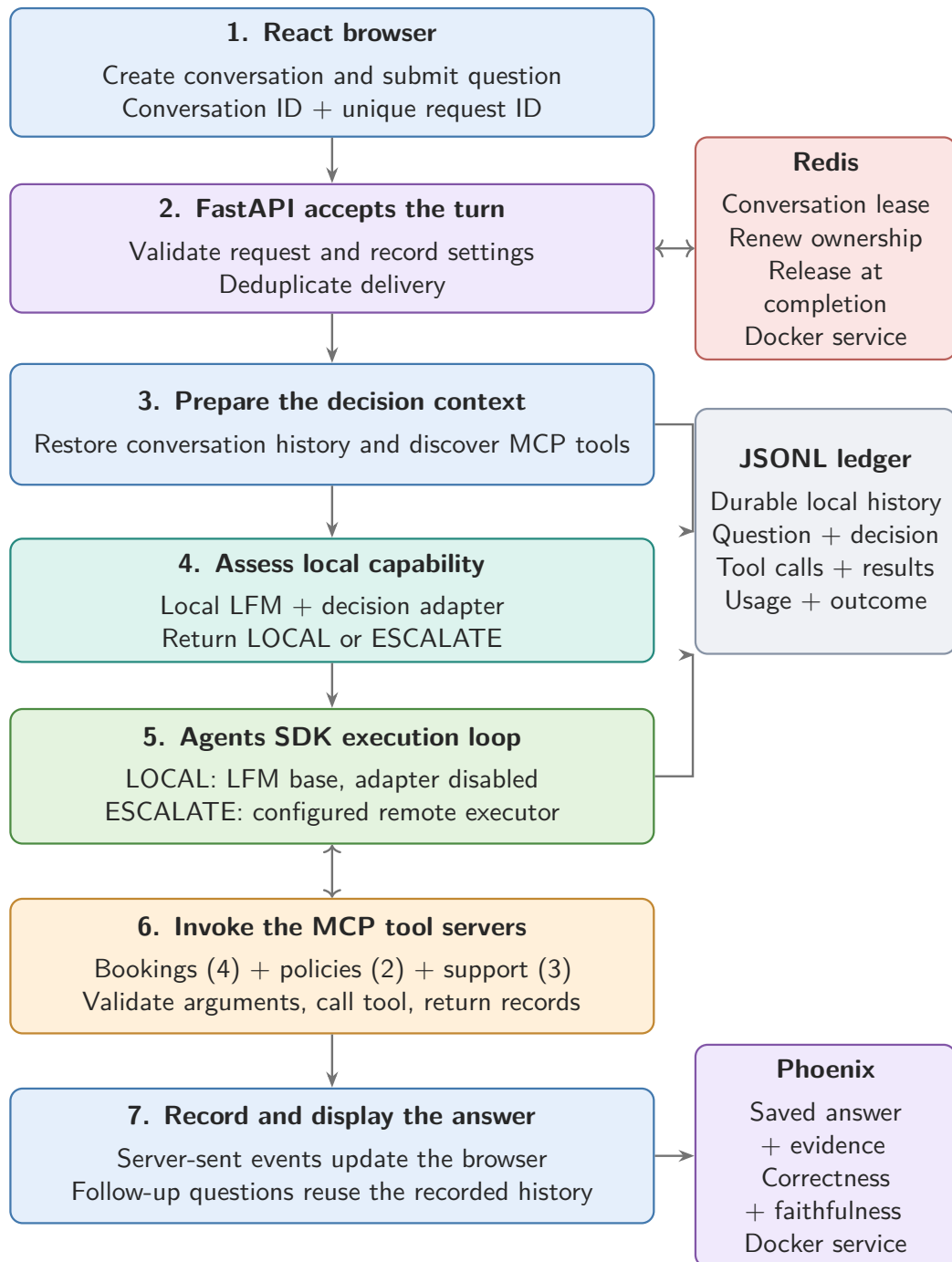


Figure 4.2: Request flow through the harness, with Redis coordination and durable conversation records.

messages, refund records distinguish requests from payments, and policy documents specify eligibility rules. Three MCP servers divide these sources into operational groups, as shown in Table 4.1. All three execution conditions receive the same nine read-only tools, allowing repeated comparisons against unchanged source records.

Table 4.1: MCP servers and evaluated tool interfaces.

Server	Tools	Role
Customer and bookings	<code>customer_search</code> <code>customer_get</code> <code>booking_list</code> <code>booking_get</code>	Profiles, histories and individual bookings.
Airline policies	<code>policy_search</code> <code>policy_get</code>	Policy discovery and complete rule text.
Customer support	<code>support_case_list</code> <code>support_case_get</code> <code>refund_list</code>	Cases, conversations and refund records.

Tasks are designed around direct field lookups, single-policy eligibility, refund reconciliation, chronological support reconstruction, filtering and constrained booking selection. Straightforward tasks ask for one bounded result. Hard tasks combine multiple dependencies, exceptions or calculations. These categories describe task design, while observed execution outcomes establish local reliability. All customer records are fictional.

4.3 Determining local capability

We measured local capability by running tasks through the unchanged local executor and checking the resulting answers and actions against the task’s source evidence. These observed outcomes supplied the supervision for the execution-decision adapter. The development work comprised an initial paired-model study and the airline-specific collection used for the final training run.

4.3.1 Synthetic tasks and matched execution

Figure 4.3 shows our initial capability-assessment workflow. We used Qwen3.8-max to generate synthetic task bundles across 12 capability categories and 12 business domains, including banking, travel, customer support and retail. Each bundle contained a request, tool definitions, fictional initial records, supporting documents where needed, and an expected outcome. We checked schemas and used an independent source review to establish that each task could be executed and assessed consistently.

We submitted executable tasks separately to LFM2.5-2.6B Q8 and Qwen3.8-27B Q8 through the harness. Each model started from its own copy of the same MCP world and chose its own sequence of tool calls. We saved messages, tool arguments, returned records, state changes, final answers and termination status. For example, retrieving a booking and cancelling the specified excursion could be checked against both the answer and the resulting booking state.

LLM as a judge was used for anonymised A/B trajectories against the reference outcome, source evidence and task constraints. It assessed completion, factual correctness, grounding, tool/state handling and constraint compliance. We assigned **LOCAL** to reviewed local success and **ESCALATE** to reviewed local failure. The Qwen3.8-27B attempt provided a comparison of execution capability. The decision target followed the local outcome, including cases where both executors failed. We labelled explicit escalation requests directly from the policy and excluded unresolved cases from training labels.

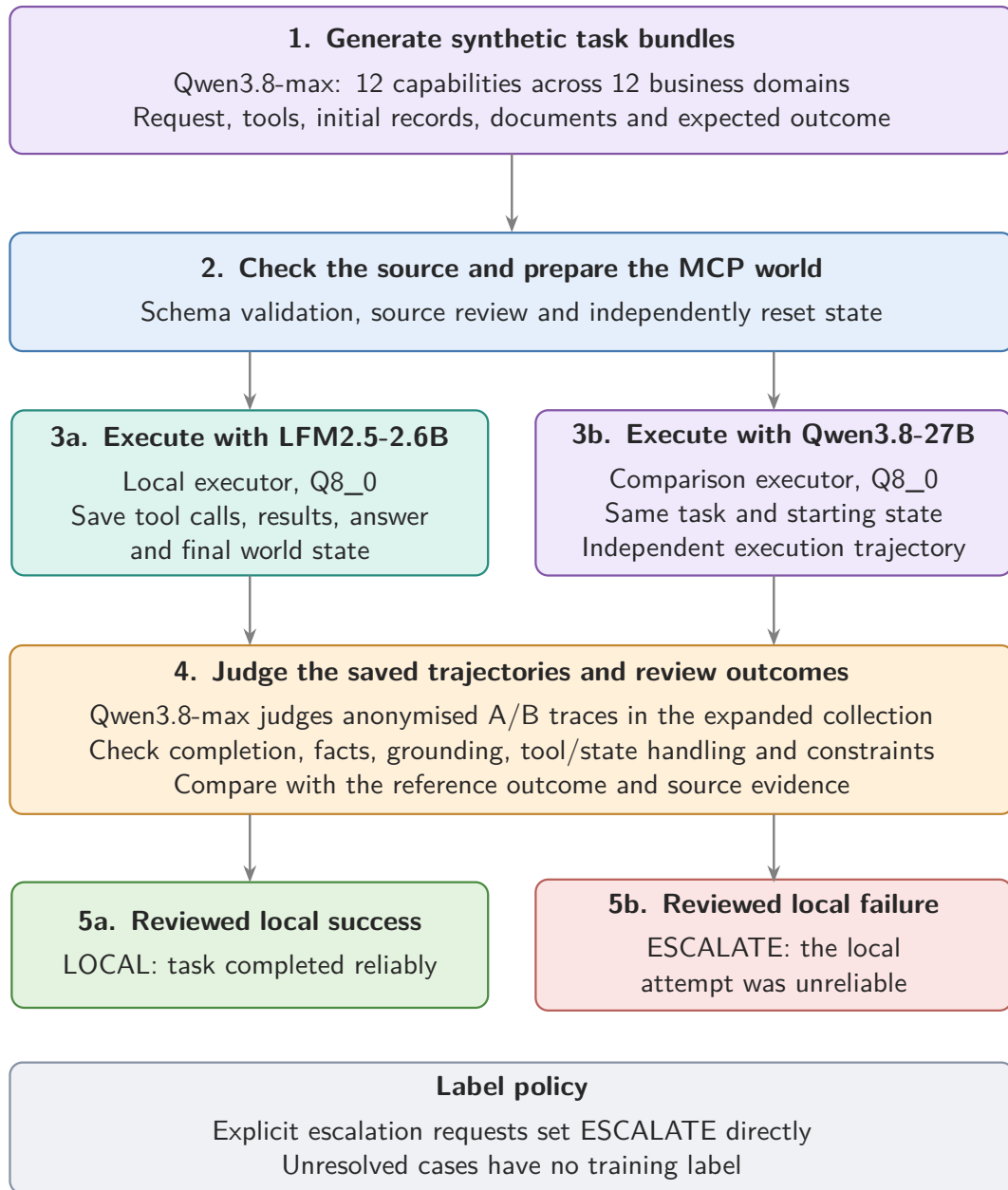


Figure 4.3: Synthetic tasks and paired execution establish observed local capability.

4.3.2 Capability labels for the final airline dataset

For the final four-epoch classifier run, we created an airline-specific set of synthetic questions covering bookings, policies, customer-support cases and refunds. The unchanged local LFM attempted each task using the available MCP tools, and its execution was reviewed against the underlying source records. Successful local execution produced a LOCAL target and confirmed failure produced an ESCALATE target. The local executor itself was not trained; its reviewed performance was used to create the classifier labels.

The airline setting provides a controlled representation of how the same method could be applied within an enterprise operating environment. Agent systems can retain execution histories containing user requests, tool calls, returned evidence and final outcomes. These trajectories can provide useful evidence about what a selected local model can complete reliably with the organisation’s own data, tools and workflows. EdgeDispatch uses this idea by turning reviewed execution outcomes into supervision for the decision model. The aim is therefore not to learn a universal notion of task difficulty, but to learn the capability boundary of a particular local executor within its intended operating environment. In another enterprise deployment, representative historical or newly collected trajectories could be reviewed in the same way to produce environment-specific LOCAL/ESCALATE training labels.

The resulting training set contains 1,020 decision contexts across 210 source conversations, with 558 LOCAL and 462 ESCALATE targets. The tasks range from direct lookups to policy application, record reconciliation, date ordering and constrained calculations, providing examples of both successful and unsuccessful local execution. A conversation can contribute multiple decision contexts because each user turn represents a new execution decision.

Each classifier input contains only the information available before execution, including the conversation and available tools. The current task’s completed answer, newly obtained tool results and reviewed outcome are used to establish the target but are excluded from its classifier input. Earlier answers and tool results remain available where they form part of the preceding conversation history.

We used separate conversation collections for training, development and prospective evaluation. The 199 resolved development contexts served as the validation set for checkpoint assessment, while the 197 resolved prospective contexts evaluated the selected classifier. Both sets were kept outside the 1,020 training examples, with each conversation belonging to only one collection. This separation avoids placing overlapping conversation histories across training and evaluation sets, since later turns can include earlier questions, responses and tool evidence. All three

collections use the same synthetic airline operating environment, allowing us to assess performance on unseen conversations within that environment.

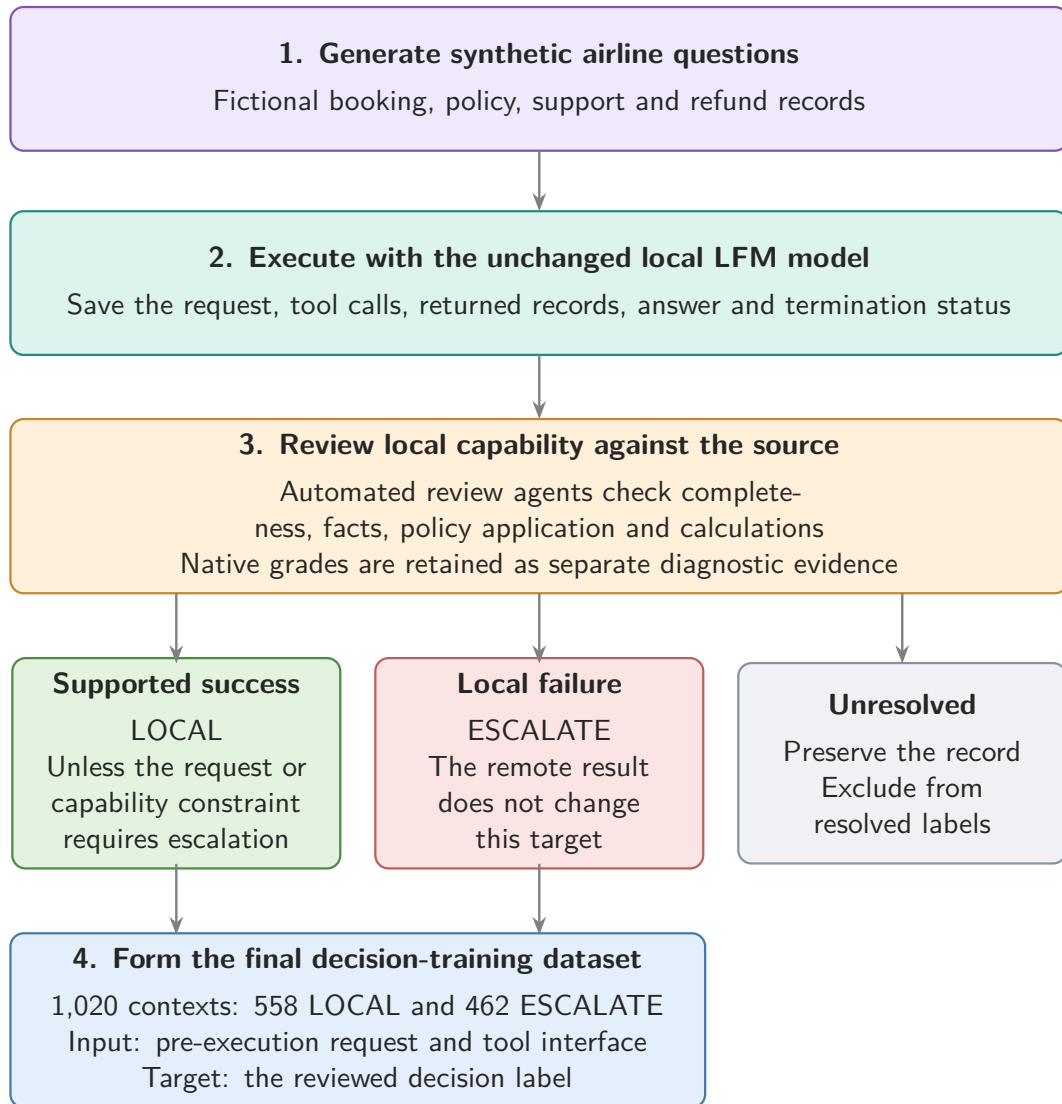


Figure 4.4: Source-reviewed LFM outcomes supply the final airline decision dataset.

4.4 SFT training procedure

We trained the execution-decision adapter for four epochs using PyTorch, Hugging Face Transformers, PEFT and safetensors with a custom training loop. The run used one NVIDIA RTX PRO 6000 Blackwell Server Edition GPU, BF16 precision, scaled dot-product attention and gradient checkpointing. We kept the base weights frozen and applied the loss only to the assistant’s label suffix, concentrating adaptation on the decision while preserving the local executor. Figure 4.5 follows this process through deployment, and Table 4.2 records the configuration. We selected the fourth-epoch checkpoint for the classifier and system evaluations.

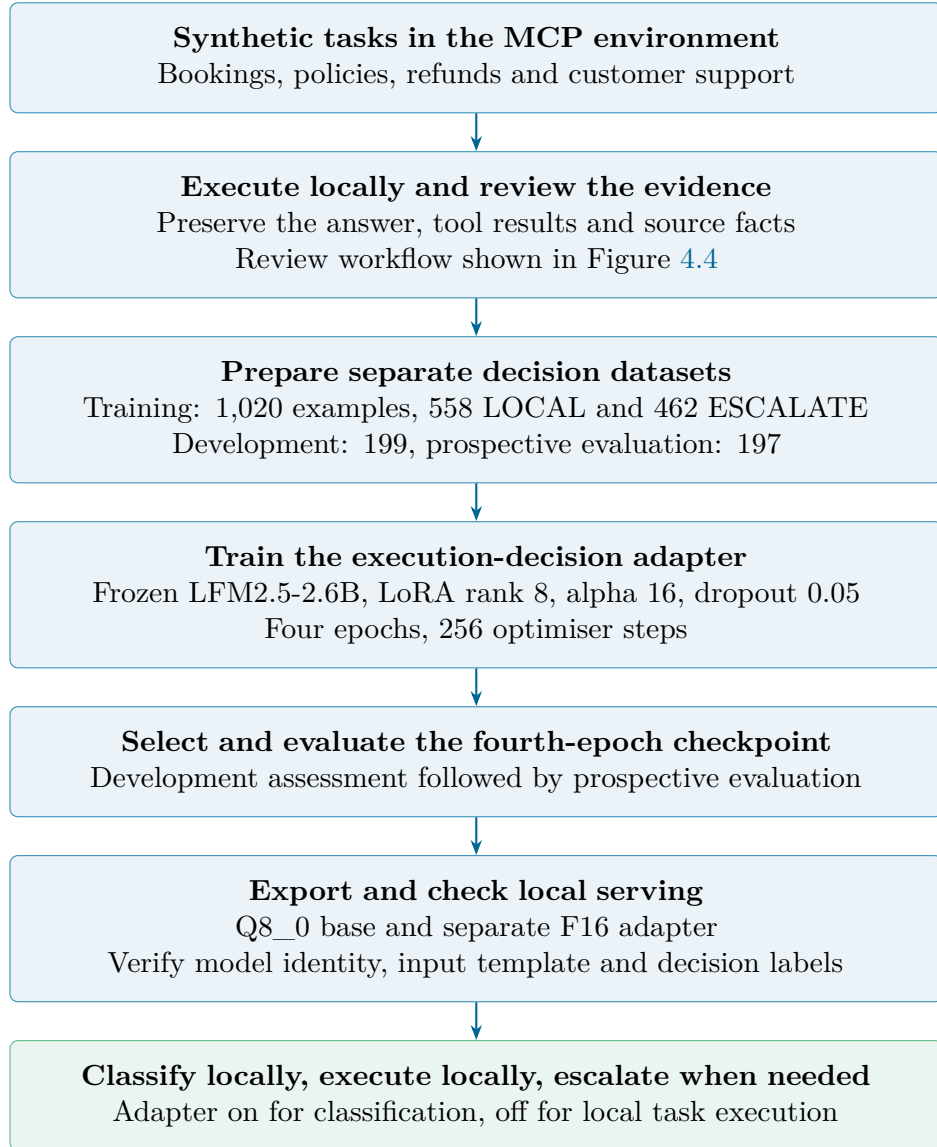


Figure 4.5: From synthetic decision examples to the deployed execution-decision adapter.

Table 4.2: Selected-run training configuration.

Setting	Value
Base model	LiquidAI/LFM2.5-2.6B, frozen weights
Training examples / epochs	1,020 / 4
LoRA rank / alpha / dropout	8 / 16 / 0.05
Target modules	w1, w2, w3, q_proj, k_proj, v_proj, in_proj, out_proj
Optimiser / learning rate	AdamW / 5×10^{-5}
Optimiser steps / presentations	256 / 4,080
Precision / random seed	BF16 / 424244
Loss	Assistant-suffix cross-entropy with class weights
Checkpoint saving	Every epoch

4.5 Prospective classifier test

After selecting the fourth-epoch checkpoint, we evaluated the classifier on a separate held-out set of synthetic airline decision contexts. The evaluation set was kept outside both training and development so that it could test whether the learned LOCAL/ESCALATE decision generalised to unseen contexts from the same operating environment.

For each context, the unchanged local executor was run with the decision adapter disabled, and its answer and tool use were reviewed against the underlying source records. A supported local success produced a LOCAL reference label, while a confirmed local failure produced an ESCALATE label. Three unresolved cases were excluded, leaving 197 labelled contexts: 86 LOCAL and 111 ESCALATE.

The selected classifier then received only the information available before execution and predicted LOCAL or ESCALATE for each context. These predictions were compared directly with the reviewed reference labels to calculate accuracy, balanced accuracy, precision, recall and F1.

4.6 Three-arm answer-quality evaluation

We submit every question independently to three conditions. Local-only uses the unchanged local executor without classifier or remote fallback. Remote-only uses `gpt-5.6-luna` directly without classifier or local fallback. EdgeDispatch first obtains the execution decision and then follows the selected path, with its existing local-failure fallback available.

We give all conditions the same source fixtures, nine tools and executor instructions. This holds the available evidence constant while comparing the execution paths.

Each item starts with one user question in a fresh conversation and may involve multiple model and tool calls.

The local context allowance is 65,536 tokens, and per-call executor output is limited to 16,384 tokens. History truncation and context shifting are disabled. Remote reasoning effort is medium and service tier is default. Local executor temperature is zero. We disable executor SDK retries and record one attempt per question and condition.

The complete hard experiment contains 100 unique hard questions, each executed once under local-only, remote-only and EdgeDispatch. This provides a focused stress test of how the three execution approaches perform on tasks that place greater demands on the executor. We then form a 50-question mixed set that provides a more varied workload in which EdgeDispatch can demonstrate both behaviours of the architecture: completing suitable requests locally and escalating more demanding requests when needed. Together, the two evaluations show performance under a concentrated hard workload and under a mixed workload where selective local execution is possible.

After execution, we assess each delivered answer with one correctness and one faithfulness judgement using Phoenix Evals. Correctness measures whether the answer fulfils the request, while faithfulness assesses its support in the retrieved evidence to distinguish task completion from grounding. We use `gpt-5.6-luna` as the judge, with medium reasoning, temperature 1, default service tier and a 4,096-token completion allowance.

For correctness, we supply the complete question, answer, available tools and recorded tool activity. For faithfulness, we supply the complete answer and retrieved context bound to tool names and arguments. We withhold execution-arm and producing-model labels from the judge and apply the same evaluator definitions across the three conditions.

Chapter 5

Results, Analysis and Discussion

5.1 Evaluation overview and task design

We evaluate EdgeDispatch in two stages. First, we test whether the classifier’s `LOCAL` or `ESCALATE` decision agrees with the reviewed outcome of local execution. Second, we compare the completed answers and remote API costs of local-only, remote-only and EdgeDispatch execution. These evaluations answer different questions: whether the system selects an appropriate execution path, and what quality and cost follow from that choice.

The answer-quality experiments use 100 hard questions and a 50-question mixed set. Each hard question is executed once under all three conditions. The mixed set combines 26 straightforward questions with 24 questions from the hard experiment, retaining their saved answers and scores. The hard set tests demanding work, while the mixed set allows us to examine both local completion and escalation. Table 5.1 illustrates the task types. Phoenix’s recorded correctness and faithfulness judgements assess the completed answers, with saved tool records and evaluator explanations supporting their interpretation.

5.2 Final classifier results

A *decision context* contains the information available at one user turn, before the classifier selects an execution path. As described in Chapter 4, we trained the adapter on 1,020 labelled contexts and used a separate **development set of 199 contexts** to assess checkpoints. We selected the fourth-epoch checkpoint after development assessment.

The held-out test set initially contained **200 contexts from 40 conversations with five questions each**. Conversations, customer groups and question-template families were kept separate from training and development. Automated review of the unchanged local executor’s attempts against source evidence established 86 successful outcomes, labelled `LOCAL`, and 111 failed outcomes, labelled `ESCALATE`. Three contexts could not be assigned a clear reference label and were excluded from

Table 5.1: Representative task types in the mixed evaluation.

Type	Question or task example	Reason for its category
Simple 1	What fare type is recorded for BOOK-0001-11?	One field in one identified booking.
Simple 2	What is the current support-case status of CASE-0022?	One case and its recorded status.
Simple 3	Is BOOK-0001-11 eligible for the eleventh-booking upgrade? Explain the decisive rule.	One booking-level decision under a named policy.
Hard 1	Audit eligible unpaid refunds across three customers and calculate the total.	Combines histories, refund payment state, policy and aggregation.
Hard 2	Reconstruct support cases chronologically and identify claims corrected by later evidence.	Requires temporal ordering and resolution of conflicting statements.
Hard 3	Choose three bookings across two customers to maximise spend within AUD 489, obeying the tie-break.	Requires a constrained optimum and comparison of equal-value solutions.

Hard examples are shortened descriptions of the complete saved prompts.

scoring, leaving **197 labelled test contexts**.

The deployed llama.cpp classifier received the current request, preceding conversation and available tools. The current question’s answer and review label were excluded. The classifier returned a valid decision for all 200 contexts. The 197 labelled test contexts were scored with reference labels using the method in Section 4.5. Table 5.2 shows the counts from which all classifier metrics are calculated.

Table 5.2: Reviewed local outcomes compared with classifier predictions.

Reviewed local outcome	Predicted LOCAL	Predicted ESCALATE	Total
Successful local execution	49	37	86
Failed local execution	11	100	111
Total	60	137	197

The matching decisions are 49 successful attempts predicted as LOCAL and 100 failed attempts predicted as ESCALATE, giving 149 correct decisions.

5.2.1 Classification measures

For this evaluation, the unchanged local model attempted the test tasks using the airline MCP tools. Automated review agents checked the saved attempts against the

original records, policy rules and required calculations. These checked results are the *reviewed local outcomes*: successful attempts received a **LOCAL** reference label, while confirmed failures received **ESCALATE**.

The classifier predicted each label using only the information available before the current question was answered. We compared its predictions with the reference labels across 197 test contexts: 86 local successes and 111 local failures. Table 5.3 reports this agreement. These results assess the execution decision, rather than the quality of the eventual local or remote answer.

Table 5.3: Classifier performance on 197 labelled held-out test contexts.

Decision	Precision	Recall	F1
LOCAL	81.67%	56.98%	67.12%
ESCALATE	72.99%	90.09%	80.65%

Overall accuracy: 75.63% (149/197). Balanced accuracy: 73.53%.

Overall accuracy measures how often the predicted decision matched the checked outcome. Balanced accuracy averages the two recall rates, giving equal weight to local successes and failures. F1 combines precision and recall into one score for each decision. The distinction between precision and recall is explained below.

5.2.2 Escalation and local retention

ESCALATE precision asks: *when the classifier chooses escalation, how often is that choice supported by a local failure?* The classifier predicted **ESCALATE** for 137 contexts, of which 100 had failed local attempts. This gives 72.99% precision. The other 37 had successful local reference attempts.

ESCALATE recall asks: *of all the tasks that failed locally, how many did the classifier identify for escalation?* There were 111 local failures, and the classifier identified 100 of them, giving 90.09% recall. The remaining 11 were incorrectly predicted as **LOCAL**.

LOCAL precision asks: *when the classifier chooses local execution, how often did the local model succeed in the checked attempt?* Of the 60 contexts predicted as **LOCAL**, 49 had successful local attempts, giving 81.67% precision. The other 11 had failed local attempts.

LOCAL recall asks: *of all the tasks that succeeded locally, how many did the classifier keep local?* There were 86 successful local attempts of which the classifier retained 49, giving 56.98% recall. The other 37 were escalated. This measures how much demonstrated local capability was retained, not how often the local model produced a correct answer.

Together, these results show stronger failure detection than local retention. This pattern is consistent with our design preference: when reliable local completion is doubtful, we favour escalation over retaining a potentially unsuccessful task. Escalating a task that could have succeeded locally incurs additional remote API cost, whereas retaining a task beyond the local executor’s capability risks an incorrect or incomplete answer. The classifier identified most local failures, while the 37 escalated successes highlight opportunities to retain more work locally. Our improvement target is therefore to expand local coverage while preserving strong failure detection, rather than maximise local retention alone.

5.3 Delivery and answer quality

On the 50-question mixed set, EdgeDispatch kept all 26 straightforward questions local and escalated all 24 hard questions. No question required an execution fallback. Table 5.4 compares this adaptive execution with local-only and remote-only execution.

Delivery is the number of final answers divided by attempted questions. **Correctness** assesses whether an answer fulfils the request, while **faithfulness** assesses whether its claims are supported by the supplied evidence. For both quality measures, we report Phoenix’s recorded passes divided by answers scored. An attempt without a final answer remains in the delivery denominator but is not scored for answer quality.

Table 5.4: Mixed-set delivery and answer quality. Quality results are judge passes among scored answers.

Question group	Execution	Delivery	Correctness	Faithfulness
Straightforward (26)	Local-only	26/26	24/26	23/26
	Remote-only	26/26	26/26	26/26
	EdgeDispatch	26/26	22/26	23/26
Hard (24)	Local-only	21/24	0/21	0/21
	Remote-only	24/24	21/24	24/24
	EdgeDispatch	24/24	24/24	24/24
Combined (50)	Local-only	47/50	24/47	23/47
	Remote-only	50/50	47/50	50/50
	EdgeDispatch	50/50	46/50	47/50

The three local hard-task non-deliveries have no final answer and are not scored. They remain in the delivery denominator.

For straightforward questions, local-only achieved 24 correctness passes from 26 answers (92.31%), compared with 22 from 26 (84.62%) for EdgeDispatch. Both used the same unchanged local executor, but their answers were generated independently.

On the hard subset, EdgeDispatch delivered all 24 answers and received 24 passes for each metric. Local-only delivered 21 answers, none of which received a correctness or faithfulness pass. The remote-only and EdgeDispatch paths also used independent executions of the same remote model, so their answers and scores could differ.

Across the mixed set, EdgeDispatch achieved **92% correctness and 94% faithfulness**, compared with 94% and 100% for remote-only. Local-only correctness was 51.06% among delivered answers (24/47), or 48% of all attempted questions (24/50). EdgeDispatch therefore improved delivery and answer quality over local-only, while achieving correctness close to remote-only on this workload and completing 52% of questions locally. Figure 5.1 summarises the correctness results.

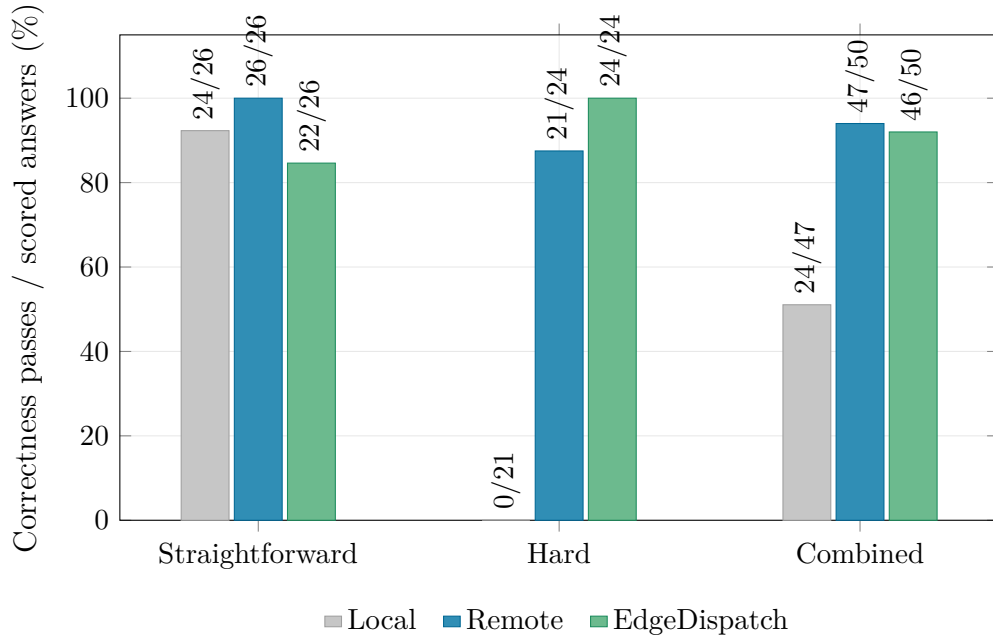


Figure 5.1: Correctness by question group. Labels show passes over scored answers. Local hard-task non-deliveries are reported in Table 5.4.

5.4 Complete hard-question experiment

The complete hard experiment contains 100 unique questions, each attempted once under all three conditions. It includes the 24 hard questions reused in the mixed set. EdgeDispatch used remote execution for all 100 questions: 99 followed valid ESCALATE predictions and one followed a safeguard for an unrecognised classifier label. We distinguish the safeguard from a valid learned decision.

Table 5.5 shows that EdgeDispatch delivered every hard-task answer, with 93 correctness passes and 94 faithfulness passes. Local-only delivered 83 answers, receiving no correctness passes and five faithfulness passes. This experiment demonstrates the role of escalation on demanding work. The mixed set and held-out classifier test

Table 5.5: Complete 100-question hard evaluation, including the 24 questions reused in the mixed set.

Execution	Delivery	Correctness	Faithfulness
Local-only	83/100	0/83	5/83
Remote-only	100/100	89/100	92/100
EdgeDispatch	100/100	93/100	94/100

Local non-deliveries comprise five emergency-deadline interruptions and twelve no-final-answer failures. Quality denominators contain scored answers only.

provide the corresponding evidence about retaining work locally. As in the mixed set, the two remote paths produced independent answers using the same configured model.

5.5 Remote execution and API-cost savings

On the mixed workload, EdgeDispatch completed 26 of 50 questions locally and sent the remaining 24 to the remote executor. Both the remote-only and EdgeDispatch conditions used `gpt-5.6-luna` (Luna) for remote execution. Compared with sending every question to Luna, EdgeDispatch achieved a **52% reduction in remotely executed questions** which resulted in nearly **14% reduction in remote execution API cost**. Table 5.6 expresses each condition’s cost relative to the remote-only baseline.

Table 5.6: Mixed-set remote use and API cost relative to remote-only execution with Luna.

Execution	Questions executed remotely	Relative execution API cost
Local-only	0%	0%
Remote-only	100%	100%
EdgeDispatch	48%	86.26%

Figures cover remote execution API charges only.

The financial benefit depends on the remote expenditure avoided by completing work locally. In this experiment, the 26 straightforward questions accounted for approximately 12.60% of the remote-only cost, while the 24 hard questions accounted for 87.40%. EdgeDispatch removed the remote execution cost of the straightforward group while retaining remote assistance for the demanding group.

Here, *straightforward* describes a request with a bounded goal, such as retrieving a booking detail or applying a named eligibility policy. Its remote cost depends

on the input and output tokens needed to complete it. In an enterprise workload, a well-defined request could involve longer records, additional tool calls or more detailed policy reasoning while remaining within the local executor’s capability.

The choice of remote executor provides another source of potential financial benefit. The reported result uses GPT Luna. In real-world enterprise deployments, using latest frontier closed source models could incur higher remote execution costs for comparable token usage, increasing the monetary value of completing suitable tasks locally. Across thousands of recurring enterprise requests, these avoided charges could accumulate into substantially larger absolute savings. The opportunity is therefore to identify valuable local work within the organisation’s operating environment.

Chapter 6

Conclusion and Future Work

6.1 Conclusions

We implemented EdgeDispatch as an Adaptive Inference Bridge that connects local capability assessment to completed work. The same LFM2.5-2.6B base supports both roles, with the LoRA adapter enabled for classification and disabled for task execution. The harness manages the conversation, calls MCP tools and returns the answer through the selected path. This gives us a working local classify-and-execute system in which decisions, source evidence, answers and API expenditure can be examined together.

RQ1: Capability-based local decisions. Our selected fourth-epoch checkpoint combines 90.09% ESCALATE recall with 81.67% LOCAL precision and 75.63% overall accuracy on the resolved prospective contexts. These results show that a compact adapter can connect the request and tool context to the assessed capability of its local executor. The two class-specific measures describe complementary achievements: identifying work to send for remote assistance and retaining work supported by successful local attempts.

RQ2: Complete answer quality. EdgeDispatch delivers every answer in the mixed evaluation and achieves 92% native correctness and 94% faithfulness. It completes the straightforward cohort locally and escalates the hard cohort, where every delivered answer receives both a correctness and a faithfulness pass. Across the complete mixed set, its correctness is close to remote-only’s 94% and substantially above local-only’s 51.06% among delivered answers. The comparison demonstrates how local completion and remote assistance can operate together within one system.

RQ3: Reduced remote use and API expenditure. EdgeDispatch completes 52% of the mixed workload locally and reduces remote execution API cost by approximately 13.74%. These results are based on a fixed 50-question workload, so the measured saving reflects the particular mix of straightforward and hard tasks used in this study. In a real enterprise environment processing thousands of requests, similar local retention could translate into substantially larger absolute savings. The experiment also uses `gpt-5.6-luna` as the remote executor. Using a more expensive

remote model, such as GPT Sol, Astra or Claude Opus 5.5, could further increase the financial value of avoiding remote execution.

RQ4: A traceable execution and evaluation workflow. The harness links each question to its decision, model execution, MCP tool results and final answer. Redis coordinates active conversation ownership, and the durable ledger retains the history used by the executor and interface. Phoenix then connects the completed answer and source evidence to correctness and faithfulness scores. We can therefore examine an individual answer alongside its supporting records and relate that evidence to the overall results.

Our central contribution is a reusable blueprint for turning observed local-model capability into an execution policy. EdgeDispatch does not assume in advance which tasks should run locally, it measures what the configured executor can complete with its available tools and context, learns a LOCAL/ESCALATE decision from those outcomes, and connects that decision directly to end-to-end task execution. The value of the approach is its reusable method: the same capability-assessment and decision-learning process can be adapted to different models, tools and operating environments. When the model, tools or operating environment change, the capability boundary can be reassessed and the decision component rebuilt from the new execution evidence. The resulting blueprint is simple and repeatable: assess the executor in its operating environment, learn where it can be trusted to act, and evaluate the quality and cost of the complete service that follows.

6.2 Future work

Enterprise hosted open models and alternative capability learning - A further direction is to examine EdgeDispatch when suitable inference hardware is not available on every employee device. Replacing or upgrading endpoint hardware across an enterprise may be impractical, particularly for models in the 30B-parameter range. An alternative is to host a larger open-weight model within the organisation’s own data centre or enterprise cloud environment using infrastructure from providers such as AWS, Azure or Google Cloud. Candidate models could include gemma-4-31B-it, Muse-Glimmer-30B, Qwen3.8-27B, or a mixture-of-experts model such as Qwen3.6-35B-A3B, where sparse activation provides more efficient inference. The same model could then support both capability classification and execution while remaining within the organisation’s controlled infrastructure.

This would test whether a larger enterprise-hosted executor can retain substantially more work before external escalation is required. The same EdgeDispatch procedure could be repeated: expose the selected model to the organisation’s tools and

representative tasks, collect its execution trajectories, review successful and failed outcomes, and use those observations to establish its capability boundary.

Existing enterprise agent systems may already retain large volumes of requests, tool calls, retrieved evidence and final responses. Rather than treating these records only as operational logs, they could provide valuable supervision for learning what the selected model can reliably complete within that organisation’s specific operating environment.

Centralised hosting introduces GPU, storage and operational costs, but these may be offset when infrastructure is shared across many users and a larger model can complete a greater proportion of requests internally. The appropriate comparison would therefore consider answer quality, inference latency, utilisation, infrastructure cost and external API expenditure together, rather than API cost alone.

A larger model also raises the question of whether SFT remains necessary for the execution decision. One approach is to continue training a LoRA decision adapter from reviewed LOCAL and ESCALATE outcomes. Another is to store reviewed trajectories and labels in a structured or vector database and retrieve relevant past examples when making a new execution decision. This would allow the system to use accumulated operational experience without retraining after every change. Comparing fine-tuned capability learning, retrieval-based capability decisions, and a hybrid of the two would show which approach best adapts as enterprise workloads, tools and models evolve.

References

- Apple (2024). Apple introduces M4 Pro and M4 Max. <https://www.apple.com/newsroom/2024/10/apple-introduces-m4-pro-and-m4-max/>.
- Apple (2025). Apple unleashes M5, the next big leap in AI performance for Apple silicon. <https://www.apple.com/newsroom/2025/10/apple-unleashes-m5-the-next-big-leap-in-ai-performance-for-apple-silicon/>.
- Arize AI (2026a). Phoenix: Correctness evaluator. <https://arize.com/docs/phoenix/evaluation/pre-built-metrics/correctness>.
- Arize AI (2026b). Phoenix: Faithfulness evaluator. <https://arize.com/docs/phoenix/evaluation/pre-built-metrics/faithfulness>.
- Chen, L., Zaharia, M., and Zou, J. (2023). FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance. *arXiv preprint arXiv:2305.05176*. <https://arxiv.org/abs/2305.05176>.
- Ding, D., Mallick, A., Wang, C., Sim, R., Mukherjee, S., Ruhle, V., Lakshmanan, L. V. S., and Awadallah, A. H. (2024). Hybrid LLM: Cost-Efficient and Quality-Aware Query Routing. *International Conference on Learning Representations*. <https://arxiv.org/abs/2404.14618>.
- ggml-org (2026). llama.cpp: LLM inference in C/C++. <https://github.com/ggml-org/llama.cpp>. Accessed 28 September 2026.
- Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., and Chen, W. (2021). LoRA: Low-Rank Adaptation of Large Language Models. *arXiv preprint arXiv:2106.09685*. <https://arxiv.org/abs/2106.09685>.
- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., Riedel, S., and Kiela, D. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *arXiv preprint arXiv:2005.11401*. <https://arxiv.org/abs/2005.11401>.
- Liquid AI (2026). LFM2.5-2.6B: Model card. <https://huggingface.co/LiquidAI/LFM2.5-2.6B>.
- Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., and Liang, P. (2024). Lost in the Middle: How Language Models Use Long Contexts. *Transactions of the Association for Computational Linguistics*, 12, 157–173. <https://arxiv.org/abs/2307.03172>.
- Lu, J., Holleis, T., Zhang, Y., Aumayer, B., Nan, F., Bai, F., Ma, S., Ma, S., Li, M., Yin,

- G., Wang, Z., and Pang, R. (2024). ToolSandbox: A Stateful, Conversational, Interactive Evaluation Benchmark for LLM Tool Use Capabilities. *arXiv preprint arXiv:2408.04682*. <https://arxiv.org/abs/2408.04682>.
- Model Context Protocol (2025a). Architecture: Specification revision 2025-11-25. <https://modelcontextprotocol.io/specification/2025-11-25/architecture>.
- Model Context Protocol (2025b). Tools: Specification revision 2025-11-25. <https://modelcontextprotocol.io/specification/2025-11-25/server/tools>.
- Ong, I., Almahairi, A., Wu, V., Chiang, W.-L., Wu, T., Gonzalez, J. E., Kadous, M. W., and Stoica, I. (2025). RouteLLM: Learning to Route LLMs from Preference Data. *International Conference on Learning Representations*. https://proceedings.iclr.cc/paper_files/paper/2025/hash/5503a7c69d48a2f86fc00b3dc09de686-Abstract-Conference.html.
- Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Zettlemoyer, L., Cancedda, N., and Scialom, T. (2023). Toolformer: Language Models Can Teach Themselves to Use Tools. *arXiv preprint arXiv:2302.04761*. <https://arxiv.org/abs/2302.04761>.
- Shi, F., Chen, X., Misra, K., Scales, N., Dohan, D., Chi, E. H., Schärli, N., and Zhou, D. (2023). Large Language Models Can Be Easily Distracted by Irrelevant Context. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202, pages 31210–31227. PMLR. <https://proceedings.mlr.press/v202/shi23a.html>.
- Yao, S., Shinn, N., Razavi, P., and Narasimhan, K. (2024). τ -bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. *arXiv preprint arXiv:2406.12045*. <https://arxiv.org/abs/2406.12045>.
- Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., and Cao, Y. (2023). ReAct: Synergizing Reasoning and Acting in Language Models. *International Conference on Learning Representations*. <https://arxiv.org/abs/2210.03629>.
- Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E. P., Zhang, H., Gonzalez, J. E., and Stoica, I. (2023). Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. *arXiv preprint arXiv:2306.05685*. <https://arxiv.org/abs/2306.05685>.